

AI-Assisted Update Guide

Cell Salts & Astrology Tracker — Expo / React Native / TypeScript
How to hand this project to other AI coding tools without breaking it.

✓ How to use this document

Paste the "Context Primer" (Section 3) into any AI coding tool — ChatGPT, Cursor, Copilot, Gemini, another Claude session, etc. — as the first message before asking it to change anything. Then follow the Verification Checklist (Section 6) after every change, no matter which tool made it.

1. Project Snapshot

Layer	Choice
Framework	Expo SDK 57 (React Native 0.86, React 19.2.3)
Language	TypeScript, strict mode (tsconfig extends expo/tsconfig.base)
Navigation	React Navigation v7 — bottom-tabs + native-stack. Entry: index.ts → App.tsx. This project does NOT use Expo Router.
State	Zustand single store (src/store/useAppStore.ts), persisted to AsyncStorage
UI system	Glassmorphic — expo-blur BlurView cards floating over a full-screen expo-linear-gradient backdrop (6 selectable presets)
Storage/Export	AsyncStorage (state) + expo-file-system's new File/Paths API + expo-print/expo-sharing (JSON/PDF export)
Notifications	expo-notifications (local, scheduled follow-up reminders)

■ Single most important fact

The bundler will sometimes print "Using src/app as the root directory for Expo Router." if expo-router happens to be installed as a dependency somewhere in the tree. Ignore that message unless you have deliberately migrated this app to file-based routing — the real entry point is index.ts → App.tsx → RootNavigator, confirmed by the bundler's own "Import stack" trace on any error.

2. Folder Map

Every file an AI tool should know about before making changes:

App.tsx	Entry point: ErrorBoundary > GestureHandlerRootView > SafeAreaProvider > GradientBackdrop + RootNavigator + DisclaimerModal
app.json	Expo config (icons, permissions, expo-notifications plugin)
src/	
components/	
BarChart.tsx	Dependency-free bar chart (analytics)
Basics.tsx	SectionHeader, Chip, EmptyState
Button.tsx	Primary/secondary/outline/danger button
Card.tsx	THE glass card (BlurView+tint) - reuse it
CellSaltListItem.tsx	Directory list row
DateInput.tsx	Typed MM/DD/YYYY field + native picker modal
DisclaimerModal.tsx	First-launch medical disclaimer
ErrorBoundary.tsx	Top-level crash fallback (class component)
GradientBackdrop.tsx	Full-screen animated gradient (renders once in App.tsx)
SeverityScale.tsx	1-10 severity picker
StarRating.tsx	5-star input
ZodiacBadge.tsx	Zodiac glyph badge
constants/	
cellSalts.ts	The 12 cell salts dataset + lookup helpers
theme.ts	LIGHT/DARK_THEME, spacing/radius/font tokens, GRADIENT_THEMES presets - the design tokens
zodiac.ts	12 zodiac signs + getZodiacSignForDate()
navigation/	
MainTabNavigator.tsx	Bottom tabs (animation: 'shift', blurred tab bar)
RootNavigator.tsx	Stack wrapping the tabs + modal/detail screens
types.ts	RootStackParamList / MainTabParamList
screens/	One screen per file, named <Name>Screen.tsx
store/	
useAppStore.ts	THE single Zustand store (profile + logs), persisted
types/	
index.ts	THE single source of truth for domain types (ZodiacSign, CellSalt, LogEntry, UserProfile, etc.)
utils/	
dateUtils.ts	Formatting helpers
exportUtils.ts	JSON/PDF export (uses expo-file-system File/Paths)
notifications.ts	Local notification scheduling
useTheme.ts	Hook: resolves colors + active gradient array

3. Context Primer — Paste This First

Copy the block below as your **first message** to any other AI tool before asking it to change code. It gives the tool the constraints it needs to avoid inventing a conflicting pattern (a second state store, a duplicate theme file, Expo Router pages, etc.).

You are editing an existing Expo (SDK 57) + React Native + TypeScript app. Follow these constraints exactly – do not deviate even if you think a different pattern is "better practice" in general:

1. Navigation is React Navigation v7 (bottom-tabs + native-stack), entry via `index.ts` -> `App.tsx` -> `src/navigation/RootNavigator.tsx`. Do NOT introduce Expo Router or a `src/app/` routing directory.
2. State lives in exactly one Zustand store: `src/store/useAppStore.ts`, persisted to AsyncStorage. Do not create a second store or move state into React Context/Redux.
3. All domain types live in `src/types/index.ts`. Extend that file; don't redefine types locally in a screen/component.
4. All design tokens (colors, spacing, radius, font sizes, gradient presets) live in `src/constants/theme.ts`. Don't hardcode hex colors or pixel values in components – import from `theme.ts`.
5. Every card/panel uses the existing `<Card>` component (`src/components/Card.tsx`), which renders a blurred, translucent "glass" surface over the app's gradient background. Reuse `<Card>` rather than writing new `BlurView` usage elsewhere.
6. Screen root containers use `backgroundColor: 'transparent'` (never a solid color) so the global gradient (rendered once in `App.tsx` via `GradientBackdrop`) shows through.
7. Read state from Zustand with a selector that returns a STABLE reference. Never do ``useAppStore(s => s.someArray.map(...))`` or ``.filter(...)/.slice(...)`` inline in a selector – that returns a new array every render and causes an infinite re-render loop ("Maximum update depth exceeded" / "getSnapshot should be cached"). Instead: select the raw field, then derive with `useMemo` in the component.
8. Any new package must be added with ``npx expo install <package>``, not ``npm install <package>`` – this resolves to the version compatible with this project's Expo SDK (57). If a package needs native code and the project is only tested in Expo Go, flag that a custom dev client / prebuild may be required.
9. `src/utils/exportUtils.ts` uses expo-file-system's NEW `File/Paths` class API (SDK 54+), not the deprecated `FileSystem.documentDirectory` string API. Keep using the new API in any related changes.
10. Run ``npx tsc --noEmit`` after every change and fix all errors before considering the change done. If possible, also run ``npx expo export --platform android`` (and ``--platform web`` if the change touches anything platform-sensitive) to confirm the bundler can build the whole module graph with no "Unable to resolve" errors.

Now here is the specific change I want: <describe your change here>

4. Golden Rules — Do / Don't

Do

- Ask for **one scoped change at a time** ("add a filter to the Tracker list") rather than open-ended requests ("improve the app").
- Ask the tool to show a **full-file rewrite** (not a fragile inline patch) whenever it touches a shared file: `theme.ts`, `useAppStore.ts`, `types/index.ts`, or any navigator.
- Ask it to **select stable references from Zustand** and derive filtered/sliced/mapped views with `useMemo` in the component body.
- Ask it to reuse `<Card>`, `<Button>`, `<Chip>`, and the tokens in `theme.ts` instead of inventing new styling primitives.
- Ask it to run **`npm expo install`** for any new native/Expo package, and to tell you if the package requires a custom dev client instead of Expo Go.
- Commit to git (or copy the project folder) **before** a large AI-driven change, so you can diff or roll back.

Don't

- Don't let a tool add Expo Router, React Query, Redux, MobX, or a second navigation/state layer "for best practice" — this app intentionally uses one navigation library and one store.
- Don't let a tool rename or move files across `src/components`, `src/screens`, `src/navigation`, `src/store`, `src/types`, `src/utils` without also updating every import — ask it to grep for all usages first.
- Don't accept a Zustand selector that returns a freshly-created array/object every call (see Golden Rule 7 in the primer) — this is the exact bug that caused the "Maximum update depth exceeded" crash during development.
- Don't let a tool bump `react`, `react-native`, or `expo` to a different major/minor version without also updating every peer dependency (`react-dom`, `react-native-web`, etc.) to a matching version.
- Don't accept solid `backgroundColor` values on screen containers — that silently hides the gradient backdrop everywhere below that screen.

5. Known Gotchas Already Hit & Fixed

If a symptom below reappears after an AI-driven change, it's a regression of a bug already solved once — point the tool at the fix.

Symptom	Root Cause	Fix
"Maximum update depth exceeded" / "getSnapshot should be cached" on web, crashes the whole nav tree	A Zustand selector returned a new array each render, e.g. <code>useAppStore(s => s.logs.slice(0,3))</code>	Select the raw array (<code>useAppStore(s => s.logs)</code>) and derive with <code>useMemo</code> in the component.
Directory tab rows overlap on phone-sized viewports	The results FlatList had no <code>flex:1</code> , and filter chips lived in nested horizontal FlatLists inside a plain column	Give the results FlatList <code>flex:1</code> ; replace nested FlatLists with plain horizontal ScrollViews for the small, non-virtualized filter rows; wrap the screen in <code>SafeAreaView</code> .
"Unable to resolve expo-blur" (or any other package) when running in an existing project	Only the <code>src/</code> folder was copied into another Expo project; the <code>package.json</code> dependencies never came along	Run <code>npx expo install</code> for every package this project's source actually imports (see Section 7 for the full list).
Bundler prints "Using src/app as the root directory for Expo Router" but the app still works	<code>expo-router</code> is present as a dependency somewhere in the tree (often from a starter template) but isn't actually the active entry	Harmless if <code>index.ts -> App.tsx</code> is confirmed in the bundler's own error "Import stack". Only a real issue if you intended to migrate to file-based routing.

6. Verification Checklist — Run After Every AI-Driven Change

- 1 **Type-check.** Run `npx tsc --noEmit` — must exit clean. This catches broken imports, wrong prop types, and typos immediately, before you ever open the app.
- 2 **Bundle-check (Android).** Run `npx expo export --platform android --output-dir /tmp/check` — confirms the whole module graph resolves (catches missing/renamed packages) without launching a device. Delete the output folder afterward.
- 3 **Bundle-check (Web),** if the change touches anything platform-sensitive (pickers, file export, blur): `npx expo export --platform web --output-dir /tmp/check-web`.
- 4 **Run it for real.** `npx expo start`, then open on the platform(s) you actually ship to. Exercise the specific screen(s) the change touched.
- 5 **Watch the console for the two known crash signatures** from Section 5 (infinite update loop / `getSnapshot` warning). If either appears, it's almost always a Zustand selector returning a new reference.
- 6 **Grep for duplicates** the tool might have introduced: a second `create(` (Zustand store), a second file exporting `ThemeColors`, or a new `src/app/` directory.

- 7 **Diff package.json.** Confirm no dependency's major version changed unless you asked for an upgrade, and that any new package was added via `npx expo install` (SDK-compatible range, not an arbitrary "latest").

✓ **Quick copy-paste for steps 1-3**

```
npx tsc --noEmit && npx expo export --platform android --output-dir /tmp/check && npx expo export  
--platform web --output-dir /tmp/check-web && echo "ALL CLEAR" || echo "FIX ERRORS ABOVE BEFORE  
CONTINUING"
```

7. Dependency Reference

Exact versions this project was built and verified against. If another AI tool needs to add a package, install it with `npx expo install <name>` so it resolves to whatever range is correct for your project's actual installed Expo SDK (which may differ slightly from the versions below if time has passed).

Package	Version	Used for
@react-native-async-storage/async-storage	^3.1.1	Zustand persist storage backend
@react-native-community/datetimepicker	^9.2.1	Native picker inside DateTimePicker's modal
@react-navigation/bottom-tabs	^7.18.18	Bottom tab navigator
@react-navigation/native	^7.3.18	Navigation container/core
@react-navigation/native-stack	^7.18.10	Stack navigator (detail/modal screens)
expo	~57.0.20	Framework
expo-blur	^57.0.2	Glass card BlurView
expo-file-system	^57.0.6	JSON export (new File/Paths API)
expo-linear-gradient	^57.0.1	Gradient backdrop
expo-notifications	^57.0.17	Local follow-up reminders
expo-print	^57.0.1	PDF export
expo-sharing	^57.0.18	Share sheet for exports
expo-status-bar	~57.0.1	Status bar styling
react	19.2.3	Core
react-dom	^19.2.3	Web target
react-native	0.86.3	Core
react-native-gesture-handler	^3.2.1	Required by navigation + App root
react-native-safe-area-context	^5.9.1	Safe-area insets
react-native-screens	^4.27.0	Native-stack/bottom-tabs dependency
react-native-web	^0.21.2	Web target
zustand	^5.0.15	State store

Dev dependencies

Package	Version
@types/react	~19.2.2
typescript	~6.0.3

npm scripts

```
npm start      expo start
npm run android expo start --android
npm run ios     expo start --ios
npm run web     expo start --web
```

8. If a Change Still Breaks Something

- 1 Copy the exact error text and the bundler's "Import stack" (if shown) back to the AI tool that made the change — don't paraphrase it.
- 2 If the tool can't resolve it in 1-2 tries, revert the change (git checkout / restore your pre-change copy) rather than letting the tool "keep trying" — accumulated speculative fixes compound into a worse mess.
- 3 Bring the reverted state, the original goal, and the error to a fresh AI session with the Context Primer (Section 3) attached — a clean context often succeeds where an increasingly confused one won't.
- 4 For anything involving native modules (camera, background tasks, etc.) that doesn't work in Expo Go, confirm whether the project has moved to a custom dev client (npx expo prebuild) — that changes what "npx expo install" alone can fix.

Generated as a working reference for the Cell Salts & Astrology Tracker project. Keep this alongside the repo and update Sections 5 and 7 whenever a new gotcha or dependency is added.